RELAZIONE PROGETTO RETI DI TELECOMUNICAZIONI

Introduzione

Il progetto si basa sulla realizzazione di un *Simulatore a eventi di un sistema a coda*.
Il modello assegnatoci è quello di una coda M/M/c, ovvero una coda in cui:

- Gli arrivi sono determinati da un processo di Poisson di intensità **λ** e spostano il processo da uno stato i a i+1.
- I tempi di servizio seguono una distribuzione esponenziale di parametro **μ**.
- Il numero di servitori presenti nel sistema è **c**.
- Quando un utente termina il servizio e lascia il sistema, il processo passa dallo stato i allo stato i-1.
- Non c'è nessun limite al numero di utenti che possono essere contenuti nella coda.

Obbiettivo

Il nostro obbiettivo è quindi quello di creare una simulazione in Java, che ci permetta di ottenere gli elementi caratteristici del sistema a coda, ovvero:

- La distribuzione di **probabilità degli stati Pk**, cioè la frazione di tempo in cui nel sistema sono presenti esattamente k clienti.
- Il **numero medio di clienti nel sistema Ls** e il **numero medio di clienti nella coda Lq**.
- Il **tempo medio di permanenza nel sistema Ws** e il **tempo medio di permanenza nella coda Wq**.
- La **probabilità di coda**, ovvero la probabilità che un cliente trovi tutti i servitori occupati e debba attendere in coda.

Una volta ottenuti i parametri che ci interessano per verificare l'efficacia della nostra simulazione gli andremo a comparare con i dati teorici e costruiremo dei grafici con un programma scritto in python.

Metodo

Per costruire questo progetto abbiamo suddiviso lo schema generale di ciò che volevamo creare in singoli componenti, in modo da poterci suddividere il carico di lavoro e semplificare la realizzazione del tutto. Siamo partiti quindi con la definizione del modello teorico (fondamentale per creare una simulazione che rispetti la teoria), poi siamo passati alla struttura del simulatore, e in fine alla validazione dei dati ottenuti.

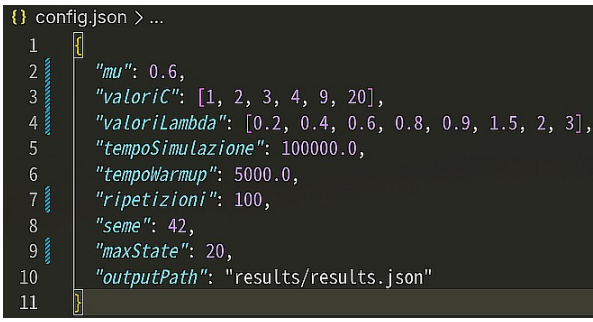
-Modello Teorico:

Foto modello teorico

-Simulatore:

Il simulatore è a eventi discreti, il tempo infatti non scorre linearmente ma avanza da un evento all'altro. La nostra simulazione conosce solo due eventi, ARRIVO e PARTENZA , e facciamo uso di una lista degli eventi futuri (una PriorityQueue ordinata per tempo) che restituisce sempre l'evento più vicino.

L'esecuzione scorre in questo ordine: prendiamo il nostro file config.json, che ha questa struttura:



Lo carichiamo all'interno della classe Config, che ci permette di validare tutti i dati e interrompere la simulazione nel caso in cui ci sia una configurazione non valida.

Definiamo inoltre due liste, RisultatiRun e RisultatiScenario, questo perché noi andremo a effettuare n (ripetizioni) simulazioni per ogni coppia di valori (**λ, μ**) e c servitori, in pratica:

- Scelgo il mio scenario, ovvero i miei valori lambda, mu e c.
- Effettuo n simulazioni con questi parametri e salvo ogni simulazione nella lista RisultatiRun.
- Una volta effettuate tutte le n simulazioni per scenario, le salvo nella lista RisultatiScenario.

Così al termine dell'esecuzione il mio programma avrà tutti i risultati delle simulazioni suddivisi per scenario e ripetizione.

Dopodichè salviamo i dati ottenuti all'interno di un file chiamato risultati.json. Questo file viene letto da un programma chiamato plot.py che tramite la libreria matplotlib costruisce i relativi grafici comparando i dati simulati con quelli teorici.

-Validazione dati:

Risultati e osservazioni